

Assignment 1 - Search Based Test Input Generation

Name : Rd Pradipta Gitaya Samiadji
Student ID : 20254099
Repository Link : <https://github.com/Diptags/cs454-assignments>

1. Overview

The file `sbst.py` generates PyTest unit tests for every top-level function of a Python file. It aims to get the full **branch coverage**. The pipeline has five stages:

1. **Parse** the target file into an AST (`ast.parse`).
2. **Instrument** the AST in memory, so that every predicate reports its branch distances.
3. **Load** the instrumented module with `compile()` + `exec()`. Nothing is written to disk.
4. **Search**, for each branch target, for an input that minimises `approach_level + normalise(branch_distance)`.
5. **Write** the inputs that add coverage to `test_<target>.py`.

File	Contents
<code>sbst.py</code>	The tool: instrumentation, runtime recorder, execution, fitness, search algorithms, test output
<code>sbst_helpers.py</code>	Small stateless helpers (AST builders, and the distance of an empty range)

Usage:

```
python sbst.py examples/example1.py [--algorithm hc|avm|sa|tabu|random] [--seed N]
[--budget N] [--init N]
cd examples && pytest
```

The default is hill climbing (in code: `hc`) with seed fixed 0 and a budget of 20,000 fitness evaluations per branch target (The limit of function execution per branch target).

2. Instrumentation

2.1 General approach

I tried to instrument the **AST in memory** and never write an instrumented file to disk. An Instrumenter walks every top-level function once and does three things for each branching statement (if / elif, while, for):

1. gives it a **branch id**;
2. records its **control dependencies**, used for the approach level (Section 3);
3. **rewrites its predicate** into calls to a runtime recorder object. The recorder is injected into the module's globals under the name `__sbst_rt__`.

Each branching statement has two **targets**: (bid, True) and (bid, False). Covering every target gives full branch coverage. The modified AST is compiled and executed in a fresh namespace. The test cases are generated against the **original** module, so the instrumentation only exists while the search execution runs.

Example: foo from example1.py after instrumentation (**printed** with `ast.unparse`):

```
def foo(x: int, y: int) -> int:
    z = 0
    if __sbst_rt__.record_branch(0, __sbst_rt__.distance_compare('==', x, 42)):
        if __sbst_rt__.record_branch(1, __sbst_rt__.distance_compare('==', y, 0)):
            z = 1
        else:
            z = 0
    else:
        z = -1
    return z
```

Every `distance_*` call returns a triple (value, d_true, d_false):

- value is the real result of the predicate;
- d_true is the distance to making the predicate True;
- d_false is the distance to making it False.

`record_branch` stores both distances and the outcome that was taken, then returns the plain boolean. **The program's behaviour is therefore unchanged.**

2.2 Branch distances

With the constant $K = 1$:

Predicate	Distance when the predicate is on the wrong side
$a == b$	$d_true = a - b $; $d_false = K$
$a != b$	$d_true = K$; $d_false = a - b $
$a < b$	$d_true = a - b + K$; $d_false = b - a$
$a <= b$	$d_true = a - b$; $d_false = b - a + K$
$a > b$ / $a >= b$	mirror images of $<$ / $<=$
A and B	d_true : distance of the first False operand, + K for each operand not evaluated; d_false : minimum d_false

Predicate	Distance when the predicate is on the wrong side
A or B	d_true: minimum d_true; d_false: distance of the first True operand, + K for each operand not evaluated
not A	swap d_true and d_false
plain value if x:	d_true = K; d_false = x (for numbers)

- **Short-circuit evaluation:** the operands of and / or are wrapped in lambdas (thunks). An operand is only evaluated when Python itself would evaluate it, so a predicate like `x != 0` and `10 / x > 1` never divides by zero.
- **Predicates inside loops** are evaluated many times per run. The recorder keeps the **minimum** distance seen for each outcome during one execution.

2.3 Loops

- **while** is instrumented like **if**: True means the code body is entered, False means the loop condition becomes False.
- **for**: the iterable is wrapped by `record_loop(bid, iterable)`, which yields exactly the same items.
 - (bid, True) is covered when the body runs at least once.
 - (bid, False) is covered when the iterator is exhausted.
 - If a range is empty, d_true is its distance to becoming non-empty. For example, `range(-5)` gives $0 - (-5) + 1 = 6$.

2.4 Keeping the search safe

Some inputs make the target loop forever. For example, in `example2.py` the input `a=300, b=400, c=0` makes neither branch inside the `while` change `a`. To prevent this from blocking the search:

- **Step limit.** Every predicate evaluation and loop iteration counts as a step. A run is aborted after 100,000 steps, or after 0.5 s (via `SIGALRM`). The limit is high enough for loops of tens of thousands of iterations; a branch that is only reached after about 15,000 iterations needs roughly 30,000 steps (one for the loop, one for the `if` inside it).
- **Aborted runs are never written as tests.** Their recorded distances are still used for fitness.
- **Exceptions** raised by the target (e.g. `ZeroDivisionError`) are caught. Such an input becomes a test using `pytest.raises`.
- Output printed by the target (e.g. in `example6.py`) is suppressed during the search.

3. Approach level

3.1 Control dependencies

While walking the AST, the instrumenter keeps a list of **control dependencies**: (branch id, required outcome) pairs that must hold to reach the current statement. Each registered branch stores this list, ordered from outermost to innermost.

- **Nesting**: the body of if/while/for adds (bid, True), and the else part of an if adds (bid, False). Because elif is an if inside the else part, it automatically depends on all previous conditions being False.
- **Early exits**: if the body of an if always leaves the block (return, raise, break, continue), then the statements *after* that if can only be reached when it was False. They therefore get the extra dependency (bid, False). Symmetrically, if only the else part always leaves, the following statements get (bid, True).

The early-exit rule matters for example3.py, where the else part of if u2 != 0 contains:

```
if u1t == 0:
    return True
if u2t == 0:      # only reached when u2 == 0 AND u1t != 0
    return True
```

Resulting dependencies for example3.py:

Branch	Line	Predicate	Dependencies
0	11	u2 != 0	–
1	15	0 <= u1	(0, True)
2	16	u1 <= 1	(0, True), (1, True)
3	17	0 <= u02	(0, True), (1, True), (2, True)
4	18	u02 <= 1	(0, True), (1, True), (2, True), (3, True)
5	22	u1t == 0	(0, False)
6	24	u2t == 0	(0, False), (5, False) ← from the early-exit rule

3.2 Computing the approach level and the fitness

For a target (bid, outcome), the **path** is dependencies + [target]. After running an input, I walk the path **from the inside out** and stop at the deepest node the execution actually reached (i.e. a node whose predicate was evaluated at least once):

- **approach level** = the number of path nodes after that node;
- **branch distance** = that node's recorded distance towards the outcome the path requires.

$$\text{fitness} = \text{approach_level} + d / (d + 1)$$

Normalising with $d / (d + 1)$ keeps the distance part below 1. This guarantees that getting one condition deeper is always better than any improvement in distance. The fitness is 0 exactly when the target is covered by a run that was not aborted.

Worked example: target "*y == 0 is True*" in foo (path: *x == 42 True* → *y == 0 True*):

Input	Deepest node reached	Approach level	Branch distance	Fitness
foo(10, 5)	<i>x == 42</i>	1	$ 10 - 42 = 32$	$1 + 32/33 \approx \mathbf{1.97}$
foo(42, 5)	<i>y == 0</i>	0	$ 5 - 0 = 5$	$0 + 5/6 \approx \mathbf{0.83}$
foo(42, 0)	Target reached	–	–	0

4. Search Approach

The search runs **one function at a time**:

1. **Random warm-up.** 10 random inputs are evaluated first, which usually covers the easy targets.
2. **Target order.** Each remaining target is attacked one by one, shallowest first (fewest dependencies). Shallow targets often cover deeper ones along the way.
3. **Collateral coverage.** Every execution records *all* the targets it covered, so targets that are already covered are skipped.
4. **Starting point.** For every uncovered target I remember the best input seen so far. The search for that target starts from this input instead of from a random one.
5. **Budget.** Each target gets at most 20,000 fitness evaluations. If the target is still not covered, it is reported as not covered (possibly unreachable), and the search moves on.

Hill climbing (required, the default). Steepest descent with random restarts:

- The **neighbours** of an input change one variable by ± 1 , ± 10 , ± 100 , ..., $\pm 100,000$, i.e. 12 neighbours per variable.
- The climb moves to the best neighbour until none improves (a local optimum), then restarts from a random input.
- The large steps let the climb reach far-away values (e.g. *y == 2007*) in a handful of moves.
- **Restart scale.** Random inputs are drawn from ± 10 , ± 100 or $\pm 1,000$, chosen at random. Small ranges make equalities between variables likely. Without this, `example3.py` stopped at 12/14 branches: its last two branches need parallel or degenerate lines, i.e. several coordinates being equal.

Test output.

- A greedy set cover keeps only the inputs that add coverage.
- Each test is written as `test_<function>_<index>` and checks the return value (`assert example1.foo(42, 0) == 1`), or the raised exception with `pytest.raises`.

5. Results and Fitness Function Evaluations

A **fitness evaluation** is one execution of the instrumented target function.

5.1 Default run (hill climbing, seed 0)

Target	Function(s)	Branches covered	Tests	Fitness evaluations
example1	foo, bar	8 / 8	6	87 (77 + 10)
example2	testme	8 / 8	3	150
example3	intersect	14 / 14	8	664
example4	tertiary	4 / 4	3	221
example5	collection	6 / 6	4	273
example6	match	6 / 6	4	16
example7	sum_of_even	6 / 6	2	10

All 30 generated tests pass. Coverage measured with `coverage run --branch -m pytest:`

File	Stmts	Miss	Branch	BrPart	Cover
example1.py	14	0	8	0	100%
example2.py	9	0	8	0	100%
example3.py	18	0	14	0	100%
example4.py	6	0	4	0	100%
example5.py	8	0	6	0	100%
example6.py	12	0	6	0	100%
example7.py	8	0	6	0	100%

5.2 Robustness over 10 seeds (hill climbing)

A single seed can be lucky, so I also tried to run every example with seeds 0–9:

Target	Full coverage	Median evaluations	Min	Max
example1	10 / 10	229	87	279
example2	10 / 10	116	17	550
example3	10 / 10	11,640	664	21,474
example4	10 / 10	220	200	316
example5	10 / 10	290	235	416
example6	10 / 10	60	16	148
example7	10 / 10	10	10	10

Hill climbing reaches 100% branch coverage in all 70 runs. `example3.py` is by far the most expensive: its last branches are only reachable on a plateau (`u2 == 0` together with `u1t == 0` or `u2t == 0`), which is escaped mostly through restarts. Seed 0 happens to be a lucky one (664 evaluations against a median of 11,640).

5.3 Same predicates, different values

I am currently assuming that the hidden test cases may use the **same kinds of predicates with different constants**. To check that the tool does not depend on the specific values in the examples, I made a variant of each example with the constants replaced by negative and larger values. `example3.py` has no constants to change, so it has no variant.

Variant	Changed predicates (original → variant)
example1	$x == 42 \rightarrow x == -987654$, $y == 0 \rightarrow y == -7$ Thus, bar uses $(x - y) / 5 > z * 3$ and $x > y + 500$
example2	$57 < c < 284 \rightarrow -300 < c < -57$, $2048 \rightarrow 5000$, $256 \rightarrow -256$, $a < 0 \rightarrow a < -10$
example4	$x == 42 \rightarrow x == -4242$, $y == 2007 \rightarrow y == 200700$
example5	$[42, 3817, 1038472] \rightarrow [-42, 3817, -1038472]$, $0 < b < 100 \rightarrow -100 < b < -50$, $\{33, 66, 99\} \rightarrow \{-33, 660000, 99\}$
example6	case 5 → case -1001, case 11 → case 99999
example7	$x \leq 0 \rightarrow x \leq -5$, $\text{range}(x) \rightarrow \text{range}(x, 10)$, $i \% 2 == 1 \rightarrow i \% 3 == 1$

With seeds 0 - 4, both hill climbing algorithm and AVM algorithm reach **100% branch coverage on every variant in every run**, and `coverage.py` confirms 100% branch coverage for the generated tests. Median fitness evaluations:

Variant	hc	avm
example1	716	128
example2	61	66
example4	461	108
example5	307	42
example6	268	91
example7	10	10

6. Extra work

6.1 Bonus constructs

All three bonus constructs from the assignment are supported. Each example reaches 100% branch coverage on all 10 seeds.

Ternary expressions in predicates (example4.py).

- **Problem:** treating `True if x == 42 else False` as a plain value gives only True/False and no gradient.
- **Solution:** the three parts (C, A, B of `A if C else B`) are instrumented separately, and their distances are combined "through" the condition:
 - C is False (B is used): `d_true = d_true(C) + d_true(A)`, i.e. flip C, then need A truthy; `d_false = d_false(B)`.
 - C is True (A is used): `d_true = d_true(A)`; `d_false = min(d_false(C) + d_false(B), d_false(A))`.
- Both sides are evaluated so that both directions have a gradient. With `x = 10` this gives `d_true = 32`, which points the search towards `x = 42`.

in operator (example5.py).

- `d_true` = the distance to the **closest element**, `min |a - e|`, and `d_false = K`.
- An empty collection gives `d_true = K`.
- Before this, the search was blind on `a in [42, 3817, 1038472]` and reached full coverage in only 1–2 of 5 seeds, using ~20,000 evaluations. Afterwards it reaches full coverage on 10 / 10 seeds with ~290 evaluations.

match-case (example6.py). A match statement has no single test expression to replace, so it is rewritten into an equivalent if/elif/else chain before instrumentation:

```
if __sbst_rt__.record_branch(0, __sbst_rt__.distance_compare('==', y, 5)):
    print('hit 5!')
elif __sbst_rt__.record_branch(1, __sbst_rt__.distance_compare('==', y, 11)):
    print('hit 10!')
elif __sbst_rt__.record_branch(2, __sbst_rt__.distance_or(
    lambda: __sbst_rt__.distance_compare('==', y, -57),
    lambda: __sbst_rt__.distance_compare('==', y, 43))):
    print('really?')
else:
    print("you haven't done anything yet")
```

- Value patterns become `==`, `|` patterns become `or`, and `case _` becomes `else`.
- The `elif` chain gives each case the correct dependencies automatically: case 2 depends on cases 0 and 1 being False.
- A match containing patterns that cannot be rewritten without changing its meaning (guards, capture patterns, sequence or class patterns) is left unchanged. While testing, I found that rewriting such patterns naively changed the program's result (e.g. returning 1 instead of 1005), which would have produced wrong assertions.

6.2 Additional local search algorithms

I implemented all three suggested algorithms. They can be selected with `--algorithm`:

- **AVM (avm, Alternating Variable Method).**
 - Optimises one variable at a time. Exploratory moves of ± 1 are followed by pattern moves whose step doubles (2, 4, 8, ...) while the fitness improves.
 - It moves to the next variable when neither direction helps, and restarts when no variable helps.
- **Simulated annealing (sa).**
 - Evaluates one random neighbour per step and accepts a worse one with probability $\exp(-\Delta / T)$.
 - T starts at 1.0, is multiplied by 0.995 per step, and the search restarts when $T < 0.01$.
- **Tabu search (tabu).**
 - Always moves to the best non-tabu neighbour, even a worse one. The last 50 visited inputs are tabu, unless a tabu input would be the best seen so far (aspiration).
 - Restarts after 5 moves without a new best.
- **Random search (random):** baseline without guidance.

Comparison (7 examples $\times$ 10 seeds, 20,000 evaluations per target). Each cell shows runs with full coverage / median fitness evaluations:

Target	hc	avm	sa	tabu	random
example1	10/10 · 229	10/10 · 47	10/10 · 908	10/10 · 229	2/10 · 20,324
example2	10/10 · 116	10/10 · 81	10/10 · 574	10/10 · 188	10/10 · 56
example3	10/10 · 11,640	10/10 · 700	9/10 · 7,994	4/10 · 22,226	10/10 · 1,428
example4	10/10 · 220	10/10 · 48	10/10 · 1,032	10/10 · 220	0/10 · 20,314
example5	10/10 · 290	10/10 · 42	9/10 · 1,472	10/10 · 290	2/10 · 21,176
example6	10/10 · 60	10/10 · 24	10/10 · 200	10/10 · 60	10/10 · 459
example7	10/10 · 10	10/10 · 10	10/10 · 10	10/10 · 10	10/10 · 10

Summary over all 70 runs	hc	avm	sa	tabu	random
Runs with full coverage	70 / 70	70 / 70	68 / 70	64 / 70	44 / 70
Branch coverage	100%	100%	99.6%	98.8%	95.0%
Mean evaluations per run	1,720	157	2,634	2,984	8,737
Total evaluations	120,396	10,958	184,376	208,861	611,592

Observations:

- **AVM outperforms hill climbing on every example.** Coverage is the same (100% in all 70 runs), but it needs **about 11× fewer fitness evaluations in total** (10,958 vs 120,396).
- On `example3.py` the median drops from 11,640 to 700.
 - Hill climbing evaluates all 12 neighbours of every variable at every step.
 - AVM only tries ± 1 on one variable and then doubles its step. Going from $x = 10$ to $x = 42$ costs about 8 evaluations.

- **Tabu search:** Somehow behaves like hill climbing on most examples, but struggles on `example3.py` (4/10). For now I still don't know how to resolve this issue.
- **Random search:** Interestingly, random search is cheaper than hill climbing on `example3.py`. Its hard branches depend on coincidences (degenerate lines) rather than on a gradient.

7. What I Learnt So Far

- Before this assignment, I thought modifying code meant editing source files. I learnt that Python's **ast** module lets you rewrite a program's structure in memory, such as adding measurement calls around every predicate, without touching the original file or changing what the program does.
- I learnt the deeper concept of SBST, even though it's quite abstract but I think I can still follow it most of the time.
- SBST implementation is really challenging, because I need to think how to define the instrumentation and approach level from different cases, in this case I have to take a look at all 7 example files
- Some inputs caused the target function to loop indefinitely. My first instinct was to add a timeout, but timeouts are imprecise and platform-dependent (In this case I am using `SIGALRM` because I am using MacOS), but turns out there is another way, such as adding the condition to abort after 100,000 steps / retries also useful.